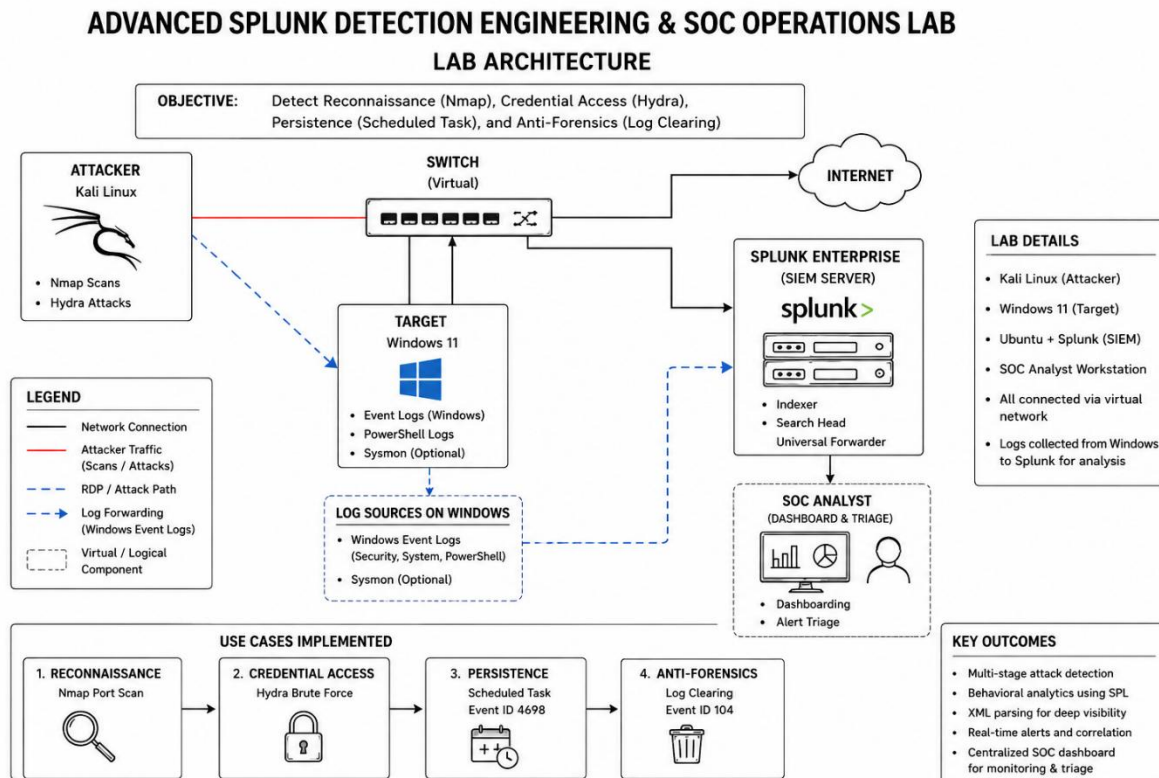


Advanced Behavioral Detection Engineering & SOC Dashboarding

Author: Qadoos Tariq Bhat

Bachelor of Computer Science (Hons.) in Cybersecurity

Objective: Develop behavioral detection use cases and centralized SOC dashboards using SPL and XML parsing techniques to identify reconnaissance and persistence activities.



Executive Summary

This project focused on behavioral detection engineering and SOC dashboard development within Splunk Enterprise. By simulating reconnaissance and persistence activities, custom detection logic was created to identify attacker behavior through log analytics and Security Operations Center monitoring workflows.

The project leveraged advanced Splunk Search Processing Language (SPL) techniques and XML parsing to transform complex event data into actionable security intelligence while providing centralized operational visibility through a custom SOC dashboard.

Technologies Used

- Splunk Enterprise
- SPL (Search Processing Language)
- XML Parsing
- Windows Event Logs
- Kali Linux
- Nmap
- PowerShell

Frameworks & Methodologies

- Cyber Kill Chain
- Detection Engineering
- Threat Hunting
- SOC Operations

Skills Demonstrated

- Behavioral Analytics
- SPL Development
- Dashboard Engineering
- Threat Hunting
- Detection Engineering
- Log Analysis
- Security Monitoring

Detection Use Cases

Reconnaissance Detection

- Nmap Detection
- Port Scan Monitoring
- Distinct Count Analysis

Persistence Detection

- Event ID 4698
- Scheduled Task Detection
- XML Parsing
- Suspicious Task Identification

Key Achievements

- Developed behavioral detection rules
- Built custom SPL analytics
- Implemented XML-based detection logic
- Engineered SOC dashboards
- Reduced investigation complexity through centralized visibility

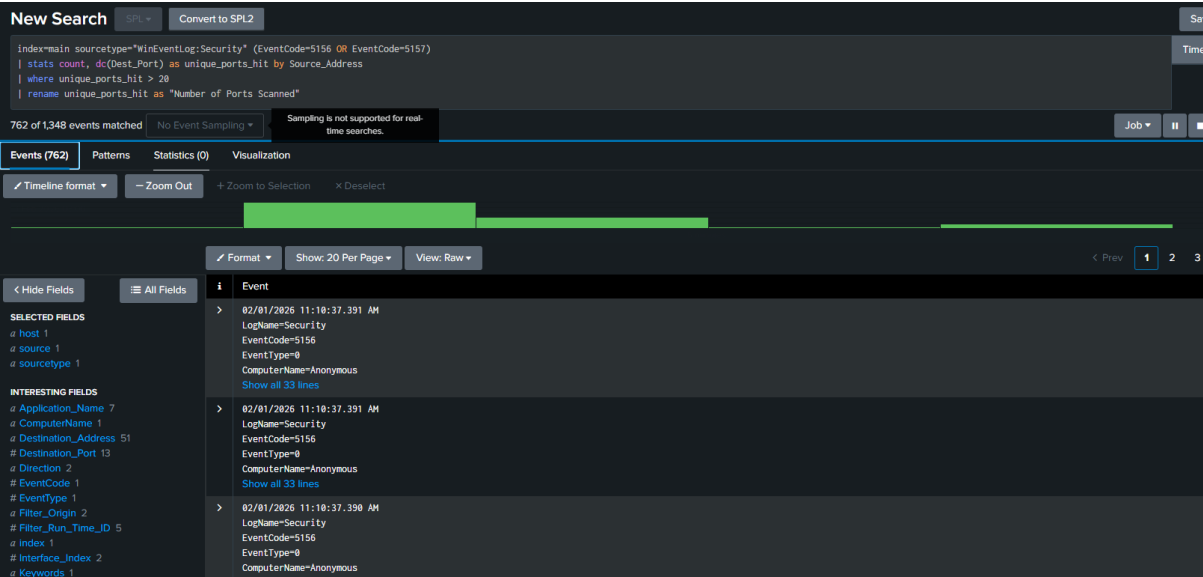
Passive Reconnaissance & Stealth Network Mapping

```
(kali@kali)-[~]
$ sudo nmap -sS -Pn 192.168.226.128
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-31 22:08 EST
Nmap scan report for 192.168.226.128
Host is up (0.00055s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
3389/tcp   open  ms-wbt-server
MAC Address: 00:0C:29:8A:FE:28 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 11.75 seconds
```

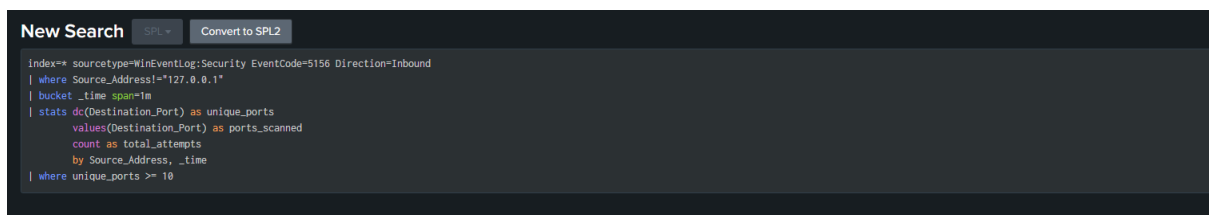
Conducted a stealth TCP SYN scan (nmap -sS) from a Kali Linux attacker machine against the Windows target. This simulates the reconnaissance phase where an adversary attempts to identify open services and potential entry points (e.g., RDP on port 3389 or SMB on port 445).

Behavioral Query Engineering & Ingestion Analysis



Developed a custom SPL (Search Processing Language) query to identify scanning behavior. By utilizing the dc() (Distinct Count) function on Dest_Port, I successfully isolated the attacker's IP address based on the volume of unique ports targeted within a short timeframe, effectively filtering out normal network noise.

Detection Logic Definition & Use Case Development



We develop detection use case of an Nmap port scanning attempt using SPL query.

Correlation Rule Implementation & Severity Calibration

The screenshot displays the 'Edit Alert' configuration window for an alert named 'RECON - Nmap Port Scan Detected'. The window is divided into several sections for configuring the alert's behavior and actions.

- Settings:** The alert name is 'RECON - Nmap Port Scan Detected'.
- Description:** The description is set to 'Optional'.
- Search:** The search query is the same SPL query as shown in the previous image, used to detect Nmap port scans.
- Alert type:** The alert is configured as 'Scheduled'.
- Expires:** The alert expires after '1' minute(s).
- Trigger Conditions:** The trigger is set to 'Per-Result'.
- Throttle:** The throttle option is unchecked.
- Trigger Actions:** A '+ Add Actions' button is present.
- When triggered:** The action is 'Add to Triggered Alerts' with a severity of 'Critical'.

At the bottom of the window, there are 'Cancel' and 'Save' buttons.

Successfully configured the 'RECON - Nmap Port Scan Detected' real-time alert.

Live Exploitation Validation & Rule Testing

```
(kali㉿kali)-[~]  
$ nmap -p- -T4 192.168.226.128  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-01 01:57 EST  
Nmap scan report for 192.168.226.128  
Host is up (0.00025s latency).  
Not shown: 65522 closed tcp ports (reset)  
PORT      STATE SERVICE  
135/tcp    open  msrpc  
139/tcp    open  netbios-ssn  
445/tcp    open  microsoft-ds  
3389/tcp   open  ms-wbt-server  
5040/tcp   open  unknown  
7680/tcp   open  pando-pub  
49664/tcp  open  unknown  
49665/tcp  open  unknown  
49666/tcp  open  unknown  
49667/tcp  open  unknown  
49669/tcp  open  unknown  
49670/tcp  open  unknown  
49671/tcp  open  unknown  
MAC Address: 00:0C:29:8A:FE:28 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 38.27 seconds  
  
(kali㉿kali)-[~]  
$ nmap -p- -T4 192.168.226.128  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-01 01:59 EST
```

Now I was using Nmap on Kali Linux to scan the ports of the Windows Machine and the ports were open.

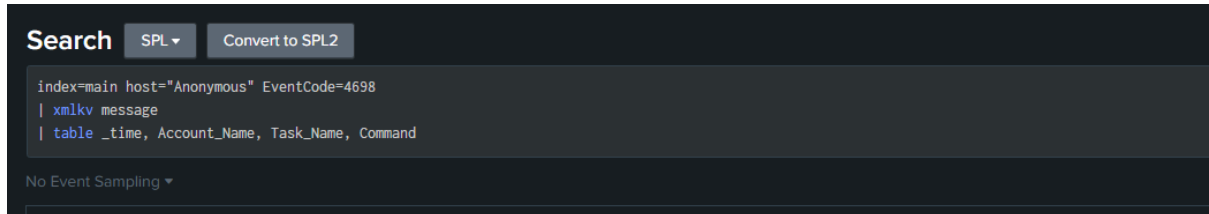
Reconnaissance Alert Verification

Filter	App	Search & Report	Owner	All owners	Severity	All severity	Alert name	All alerts	Showing 1 - 1 of 1 results	Show: 20 per page	1	of 1 pages
☐	Time		Alert name		App	Type	Severity	Mode	Actions			
☐	2026-02-01 07:00:24 UTC		RECON - Nmap Port Scan Detected		search	Real-time	Critical	Per Result	View Results Edit Alert Delete			

As we can see the alert was successfully triggered.

Persistence Engineering & Structured XML Parsing

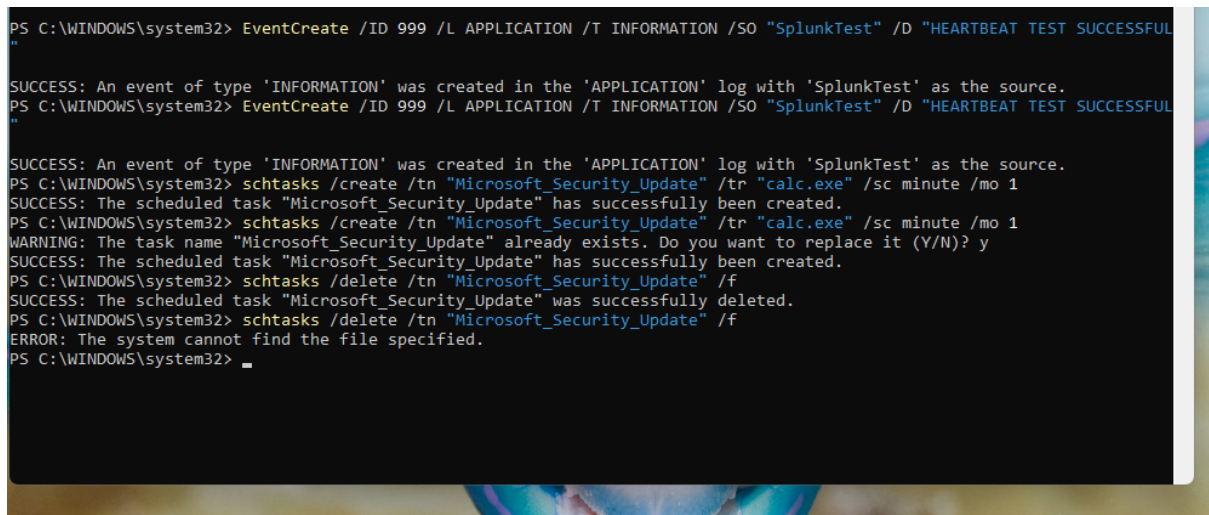
Now let's try another alert for Persistence detection.



In this phase of the lab, I focused on detecting 'Persistence'—the stage where an attacker attempts to maintain a foothold in the system. I built a custom detection rule for **Windows EventCode 4698** (Scheduled Task Creation). Since Windows stores the most critical details of this event within a complex XML blob, I utilized the **xmlkv** command to programmatically extract the **Task_Name** and **Command** fields. This allowed me to transform raw, unreadable logs into actionable intelligence, specifically flagging a suspicious task I simulated (**Microsoft_Security_Update**) that was set to execute a hidden payload.

Adversarial Emulation & Payload Scheduling

Then i created an alert on the basis of this detection logic and enabled it .



Then I used this command in the windows powelshell .

Persistence Alert Velocity Verification

☐	Time	Alert name	App	Type	Severity	Mode	Actions
☐	2026-02-01 09:56:25 UTC	RECON - Nmap Port Scan Detected	search	Real-time	Critical	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:56:19 UTC	RECON - Nmap Port Scan Detected	search	Real-time	Critical	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:56:05 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:55:43 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:55:12 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:54:15 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:53:54 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:53:22 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:52:26 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:52:04 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:51:33 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:50:38 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:50:14 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:49:44 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:48:46 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:48:25 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:47:54 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:46:57 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:46:36 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete
☐	2026-02-01 09:46:04 UTC	PERSISTENCE - New Scheduled Task Created	search	Real-time	High	Per Result	View Results Edit Alert Delete

As we can see the alerts were triggered successfully.

Single-Pane-of-Glass SOC Dashboard Integration

The screenshot shows a Splunk Enterprise dashboard with the following components:

- Header:** Splunk Enterprise logo, navigation tabs (Search, Analytics, Datasets, Reports, Alerts, Dashboards, Modules), and user information (Administrator, Messages, Settings, Activity, Help).
- Dashboard Title:** Network Reconnaissance & Behavioral Analysis. Subtitle: Advanced detection for high-velocity port scanning and footprinting.
- Left Panel (NMAP Info Gathering):** A table with columns 'Time' and 'Event'. It displays three events from 02/02/2026 12:23:00.000 to 12:23:12.189 PM. Each event includes details like LogName=Security, EventCode=5156, EventType=0, and ComputerName=Anonymous. The source is WinEventLog:Security.
- Right Panel (Host-Level Persistence: Scheduled Task Creation):** A table with columns '_time', 'host', 'Suspicious Task', 'Payload Path', and 'Executed User'. It displays a list of events showing suspicious tasks like 'Microsoft_Security_Update' running as 'calc.exe' on 'Anonymous' hosts.

I engineered a centralized **Security Operations Center (SOC) Dashboard** to bridge the gap between network telemetry and host-based forensics. This dashboard serves as a 'Single Pane of Glass' for incident responders, correlating multi-stage attack data in real-time. By implementing custom SPL filters to isolate high-velocity port scans alongside XML-parsed scheduled task events, I have created a functional monitoring environment that drastically reduces the time required to identify and triage a breach.

Conclusion

With a fully functional Splunk environment capturing and visualizing the 'Reconnaissance' and 'Persistence' phases of an attack, I have established a solid baseline for SIEM operations. To further expand my security engineering capabilities, this implementation playbook successfully bridges the gap between raw network mapping and advanced host forensic telemetry.

By deploying precise SPL logic using specialized commands like `dc()` and `xmlkv`, the environment has been moved away from brittle signature matching and toward robust behavioral detection engineering. The resulting live alerts and centralized dashboard panels demonstrate a deep engineering competency in identifying reconnaissance patterns and persistence mechanisms, laying down a highly scalable framework for modern threat triage and automated enterprise monitoring.